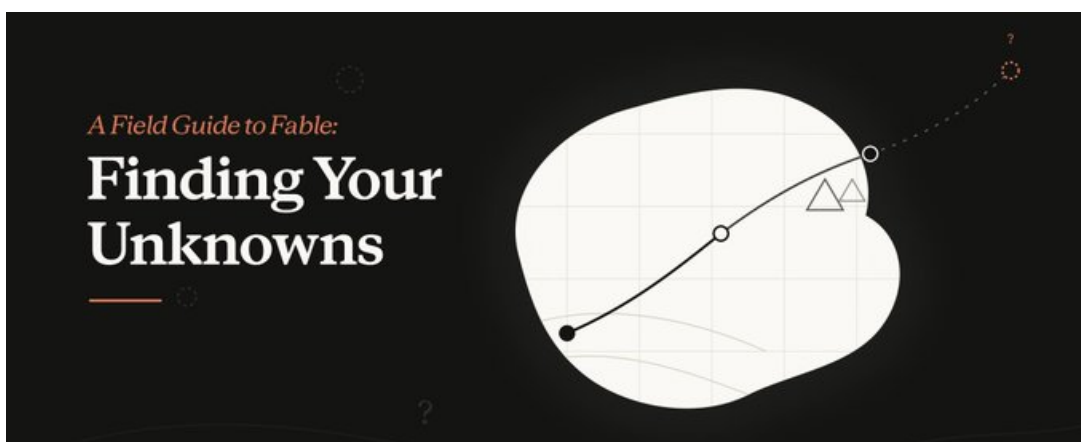


把未知变成可审阅工件

从 Know your unknowns 到 HTML Effectiveness

本地资料整理

2026 年 7 月 6 日



资料来源：本地 HTML 镜像与补充网页正文

离线入口：source/html_mirror/START_HERE.html

原始站点：<https://thariqs.github.io/html-effectiveness/>

Unknowns 子站：<https://thariqs.github.io/html-effectiveness/unknowns/>

说明：文中的 Acme、Fable 发布语境和示例工程均按本地材料处理；案例用于解释工作法，不作为外部事实背书。

目录

导读

本文的核心结论是：长程代理工作流的质量，取决于能否把未知尽早转化为可审阅工件。补充网页提供四类未知、地图与地形、实施前到实施后的方法框架；本地 HTML 示例则展示这些方法如何落到页面、原型、计划、实施日志、pitch 和 quiz 中。全文按一个递进链条展开：先定义未知，再解释 HTML 作为中间工件的价值，随后分别处理实施前、实施中、实施后的工件，最后收束成可复用的团队模板。

1 为什么要先知道自己的未知

本章的结论是：长程代理可以执行很多步骤，人的关键工作先变成识别缺口。提示词、计划和已知上下文只是“地图”；真实代码库、历史迁移、用户偏好、运行环境和评审标准才是“地形”。先知道自己的未知，就是先找出地图与地形之间的落差，再让代理围绕这些落差产出可检查的工件。

1.1 地图与地形：未知来自两者之间的缝隙

材料中的 `map / territory` 类比很适合解释代理工作流。地图是人已经写下的东西，例如目标、约束、候选方案、项目路径、计划步骤和提示词。地形是任务真正发生的世界，例如代码里的历史迁移、旧实现的回滚原因、团队默认审查口径、用户会实际操作的界面，以及运行时才出现的边界条件。

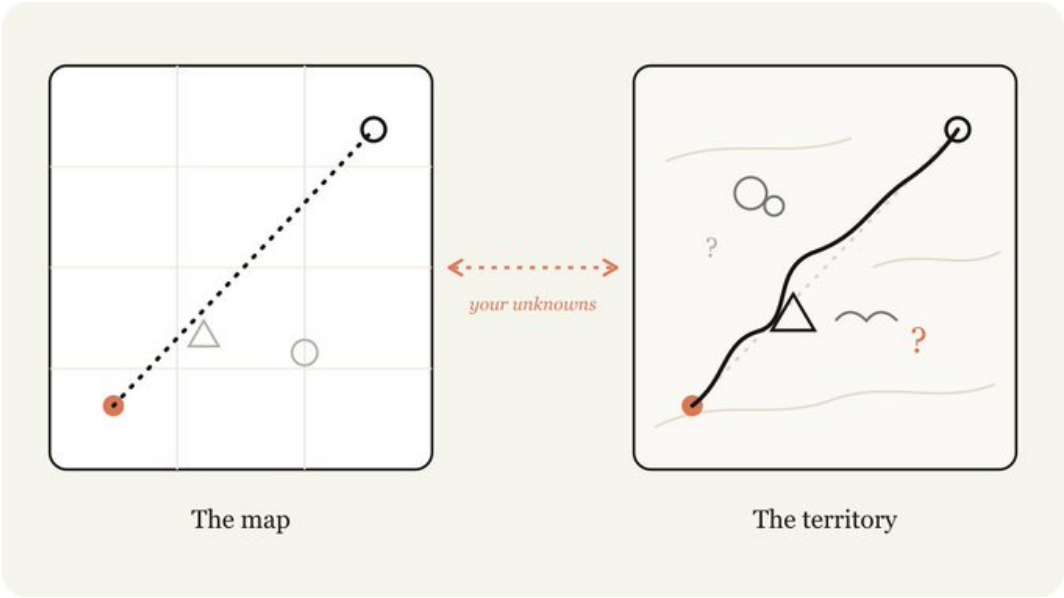


图 1: 地图与地形的差距就是 unknowns。地图可以很整洁，真实地形会包含绕路、障碍、隐含标记和没有被写进提示词的判断。

这个类比的教学价值在于：它让“写提示词”从单纯补充细节，变成一次地形侦察。实践者越早承认地图有缺口，越容易让代理先搜索、解释、模拟和提问，避免在真实实现里用高成本方式发现问题。

核心判断

在长程代理工作中，最昂贵的失败通常来自“以为地图已经覆盖地形”。更稳妥的启动方式，是先要求代理指出地图缺了哪些地形信息，再把这些缺口转化为可审阅的页面、清单、模拟或问题。

1.2 四类未知：把模糊焦虑拆成可处理对象

补充材料把 unknowns 拆成四类。这个拆法让“我不知道该怎么写提示”变成更具体的问题：哪些事实已经写下，哪些问题自己知道要问，哪些偏好只有看到方案才会显现，哪些风险当前完全没有进入视野。

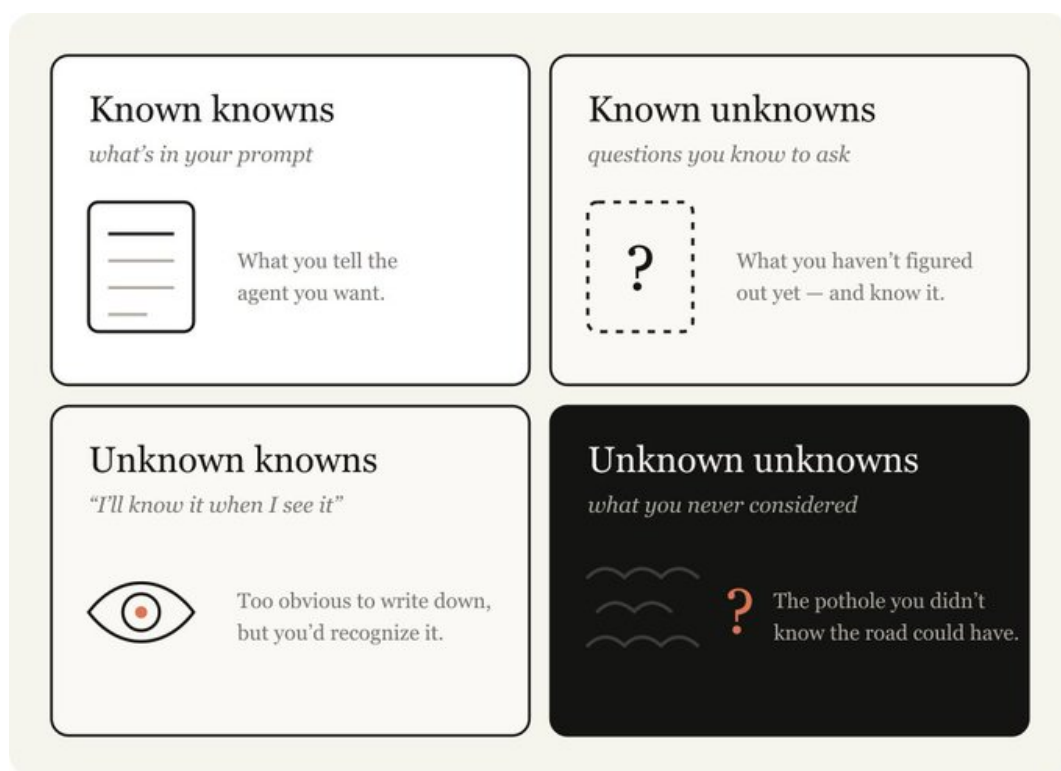


图 2: 四类未知把提示词、显式问题、隐性判断和完全盲区分开，帮助读者判断下一步该补资料、看原型、问问题，还是先做盲区扫描。

类别	含义	适合的工件
Known knowns	已经写进提示词的事实、目标和约束。	任务摘要、需求清单、实现计划。
Known unknowns	人知道自己还没决定或没理解的问题。	澄清问题、决策表、待确认清单。
Unknown knowns	人没有写下，但看到方案时能立刻判断的偏好和经验。	多方向 mock、交互原型、视觉对照页。
Unknown unknowns	人尚未意识到的盲区，例如历史回滚、隐藏迁移、权限边界。	Blindspot pass、历史扫描、风险地图。

表 1: 四类未知对应四种不同的处理动作。把它们混在一起，会让提示词变长，却仍然无法覆盖真正的风险。

四象限的关键价值是降低认知负担。已知事实适合直接写入提示词；已知问题适合让代理采访或列出选项；隐性偏好适合用多个可看的版本激发反应；未知盲区需要代理先读代码、历史和边界，再反过来改写更好的实现提示。

1.3 为什么要在实施前处理未知

实施前处理未知的成本最低，因为此时修改的对象通常只是提示词、HTML 页面、计划和解释文档。真实代码还没有被改动，迁移还没有生成，评审者也没有被迫理解一个已经成形的 diff。补充材料中的 Blindspot pass（盲区扫描）提示说明了这一点：面对陌生认证模块时，先让代理扫描 services/auth/、历史提交和迁移状态，输出七个本来会“踩进去才知道”的问题。

这个过程把“缺经验”转换为“有证据的下一轮提示”。例如，认证模块示例暴露了会话存储迁移未完成、SAML 路径绕开通用中间件、既有 Okta 尝试被回滚、身份链接模型藏在 provider 接口之后等问题。这些信息一旦写进下一轮提示，代理的实现边界就会更接近真实地形。

一个实用类比

地图像会议前写下的议程，地形像会议室里真实的人、旧账、禁区和默认规则。议程能让讨论开始，地形决定讨论能否落地。先知道未知，就是先问清哪些人会反对、哪些旧决定仍有效、哪些词在团队里有特殊含义。

1.4 隐藏假设与限制

先知道未知并不等于一次性消除风险。它依赖三个前提：代理能读取足够的本地材料；人愿意承认自己需要先反应再决定；产出的工件会被认真审阅。缺少任一前提，HTML 页面或风险清单都可能变成漂亮的旁路材料。

还需要区分示范对象和事实对象。材料中的 Acme 场景是示例案例，适合用来解释工作法；正文不能把它当作真实公司事实。Fable 相关内容也只能依据本地补充页理解，不能扩展为未经核验的外部产品结论。

常见误区

把 unknowns 当成“资料不够”会低估问题。很多 unknowns 来自偏好、历史、团队审查习惯和实现细节之间的冲突，单纯补更多文字未必能解决。更好的做法是选择能迫使这些冲突显形的工件。

1.5 本章小结

先知道自己的未知，是把代理工作流从“写更长的提示词”推进到“先暴露地图与地形的差距”。四类未知提供了分类框架：已知事实进入提示词，已知问题进入采访和决策表，隐性偏好进入可反应的 HTML，未知盲区进入扫描和风险地图。后续章节会说明为什么 HTML 特别适合作为这些缺口的中间工件。

2 HTML 为什么适合作为中间工件

本章的结论是：HTML 适合作为代理工作流的中间工件，因为很多工作成果本来就有空间结构、视觉状态、交互反馈和可复制输出。线性 Markdown 适合陈述，HTML 适合让人看见关系、点击状态、比较方案，并把反应重新带回下一轮提示。

2.1 HTML 把“读材料”变成“看结构”

html-effectiveness 根目录给出二十个自包含 HTML 示例，覆盖探索规划、代码审查、设计、原型、图示、演示、研究学习、报告和自定义编辑器。它们共同回答一个问题：当信息的形状很重要时，网页比线性文本更能保留结构。

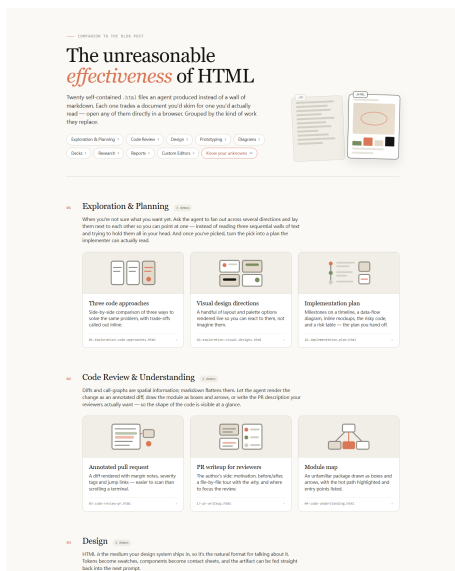


图 3: HTML Effectiveness 索引把二十个示例按工作类型分组。页面本身已经展示了 HTML 的优势：分区、卡片、标签和缩略图让案例空间可以被快速浏览。

代码审查和模块理解是典型例子。04-code-understanding.html 把认证请求路径、调用栈、关键文件和 gotchas 放在同一页中，读者可以同时看到 verifyToken、SessionStore、路由和数据库表之间的关系。13-flowchart-diagram.html 把部署流水线画成可点击流程图，让 git push main 之后哪些步骤会运行、哪些地方会短路，成为一眼可见的路径。

编号	页面主题	在本文中的背景作用
01	Debounce search 三方案	展示代码方案比较如何把取舍、依赖和复用条件并列呈现。
02	Empty state 四种视觉方向	展示视觉偏好如何通过多个可比较方向显形。
03	Optimistic update PR review	展示评审材料如何按风险、文件和下一步组织。
04	Authentication flow	展示代码理解页面如何把 trust boundary、调用栈和 gotchas 放到同一视野。
05	Design system reference	展示 token、组件和样式规则如何成为可携带提示上下文。
06	Card variant matrix	展示组件变体如何通过状态矩阵辅助选择。
07	Task completed animation	展示微交互如何用可播放原型暴露手感差异。
08	Sidebar drag-to-reorder	展示低成本交互原型如何提前检验拖拽决策。
09	Platform engineering slide deck	展示演示材料如何把进展、风险和决策压缩成会议对象。
10	Background jobs SVG illustrations	展示可导出的图示如何服务文档说明。
11	Engineering status report	展示团队状态如何以 highlights、shipped、velocity 和 carryover 组织。
12	Incident report	展示事故材料如何保留 timeline、root cause、impact 与 action items。
13	Deploy flowchart	展示流程图如何让部署路径和短路点可点击。
14	Rate limiting explainer	展示代码研究如何把请求路径、配置和 gotchas 分层呈现。
15	Consistent hashing explainer	展示抽象算法如何通过图形和对照表降低理解负担。
16	Comment threads implementation plan	展示实施计划如何按可评审切片、数据流、风险和 open questions 组织。
17	Notification queue PR writeup	展示 PR 说明如何引导评审者关注 worker、enqueue 和 rollout。
18	Cycle triage board	展示编辑器型工件如何把待办项拖拽到 Now、Next、Later 与 Cut。
19	Feature flags editor	展示配置编辑如何把依赖警告和输出 diff 放在同一页。
20	Support prompt tuner	展示提示词调整如何同时检查多个客服语气场景。

媒介判断

当任务需要比较、定位、点击、折叠、筛选、拖拽或复制结果时，HTML 的表达能力会直接影响理解质量。它把“代理说了什么”变成“人可以怎样检查和反应”。

2.2 HTML 能承载空间关系、状态和选择

很多工程判断不只依赖文字。视觉设计需要看密度、节奏、留白和层级；交互原型需要感受拖拽、点击、动画和反馈；流程图需要显示分支、阻塞点和短路路径；编辑器需要把配置变化即时反映到输出结果。

根目录示例分别覆盖了这些能力：

- 02-exploration-visual-designs.html 把同一个空状态做成四种视觉方向，帮助人通过反应表达偏好。
- 08-prototype-interaction.html 用原生拖拽模拟侧边栏重排，让“手感”成为可判断对象。
- 15-research-concept-explainer.html 用一致性哈希的环形解释和对照表，把抽象算法变成可导航的学习页面。
- 20-editor-prompt-tuner.html 把 prompt 编辑区和三张示例工单的渲染结果放在一起，让提示词质量在多个语气场景下同时被检查。

这些页面的共同点是：它们把中间判断放到浏览器里。浏览器提供空间、状态和交互；代理提供初稿；人通过点击、选择和修改，把隐性偏好转成下一轮输入。

2.3 HTML 让评审者减少工作记忆负担

线性文本要求读者把多个片段记在脑中。例如读一个 PR 描述时，读者需要同时记住动机、风险、文件列表、测试计划和回滚方案。HTML 可以把这些内容变成页面结构：风险放在醒目区域，文件按

阅读顺序排列，状态用颜色区分，流程用图形连接。

17-pr-writeup.html 展示了这种转化。它解释通知发送从请求路径迁移到队列的原因，再把文件按“先看 worker，再看 enqueue call site，再看 plumbing”的审查顺序组织。这个顺序优先服务评审者理解，字母排序只适合查找。11-status-report.html 和 12-incident-report.html 也说明，状态报告和事故复盘会受益于时间线、表格、数字卡片和颜色编码。

工作记忆的类比

线性文本像把所有零件倒在桌上，读者需要自己拼出结构。HTML 像把零件按功能分区摆好，并标出哪些可以先看、哪些需要交互、哪些是最终输出。读者仍然要判断，但不必把全部结构临时装在脑中。

2.4 HTML 是中间工件，仍需回到真实交付物

HTML 的强项是暴露未知和促成反应，交付责任仍然要回到代码、测试、文档、评审材料或最终 PDF。一个静态 HTML mock 可以证明布局偏好，却不能证明真实组件已正确接入权限、数据加载、无障碍和错误处理。一个流程图可以展示部署路径，却不能代替 CI、日志和运行时验证。

因此，HTML 工件最适合放在两个位置：一是实施前，用低成本暴露盲区和偏好；二是评审前，把复杂变更压缩成可审阅证据。它的风险也清楚：页面可能使用假数据，交互可能只模拟 happy path，样式可能看起来接近产品却没有继承真实设计系统。正文后续会把这些限制放进具体工作法。

边界条件

HTML 工件可以提高理解速度，但它不能自行保证事实正确。凡是涉及权限、数据一致性、性能、迁移、外部规格和生产行为的结论，都需要回到代码、日志、测试或明确的源材料中验证。

2.5 本章小结

HTML 适合作为中间工件，因为它能同时承载结构、视觉、状态和可复制输出。它把代理生成的解释、原型、计划、审查材料和编辑器变成可看、可点、可讨论的对象。它的主要价值是把人的反应提前放进工作流，让未知在低成本阶段显形，并继续服务后续真实实现。下一章会把这种媒介能力具体落到实施前的盲区扫描、设计反应、采访和计划排序。

3 实施前：用低成本工件暴露高成本盲区

本章的结论是：实施前最值得做的事情，是用便宜、可丢弃、可审阅的 HTML 工件暴露那些进入代码后会变贵的盲区。本文把 unknowns/01..08 视为一套方法谱系：先找地形风险，再补领域语言，再显化偏好，最后把方案、语义和决策顺序变成可确认材料。

3.1 实施前是发现 unknowns 的低成本窗口

unknowns/index.html 把实施前案例放在最前面，并明确指出：代码写下之前，是发现未知最便宜的位置。此时可以让代理扫描陌生地形、补足领域词汇、生成多个可反应版本、采访人类决策者，或把参考实现转成语义映射。

这个阶段的核心原则是：工件要便宜到可以推翻，具体到可以反应。便宜意味着它还没有触碰真实代码路径；具体意味着人看完之后能指出“这里错了”“这个方向对”“这个决策要改”“这个边界要先问清”。

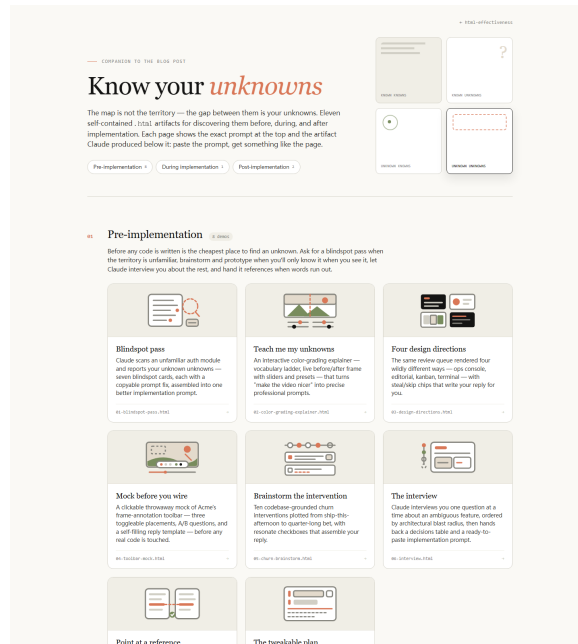


图 4: Know your unknowns 索引把实施前、实施中、实施后三个阶段分开。实施前区域包含八个 demo，覆盖盲区扫描、领域教学、设计方向、mock、脑暴、采访、参考移植和可调计划。

3.2 先处理地形风险：Blindspot pass

`unknowns/01-blindspot-pass.html` 处理的是未知盲区。示例中的人只知道自己要“添加一个新的 SSO auth provider”，但代理扫描后发现真实地形复杂得多：会话存储迁移卡在中间，SAML 路径绕开通用中间件，过去已有 Okta OIDC 尝试被回滚，身份和账号的模型也没有暴露在 provider 接口里。

这类工件的价值在于，它把“陌生代码区域”拆成可执行的下一步：先读哪些历史，避开哪些模板，验证哪些契约，哪些路径需要额外测试。对于代理 workflow 而言，这比直接要求实现更稳，因为实现者得到的是带地形约束的任务。

使用规则

当任务涉及陌生模块、历史迁移、权限边界、旧方案回滚或团队约定时，先做 **Blindspot pass**。目标是产出一份更真实的实现提示，把解决方案推迟到地形约束明确之后。

3.3 再补语言和偏好：教学页、方向页与 mock

有些未知来自领域语言不足。`unknowns/02-color-grading-explainer.html` 处理这种情况：人只能说“让视频好看一点”，代理则把工作拆成 `ingest`、`correct`、`grade`、`match`，并解释 `exposure`、`white balance`、`contrast curve`、`lift / gamma / gain`、`saturation / vibrance`、`LUT` 等词。结果是下一轮提示可以使用专业词汇，表达也更可执行。

另一些未知来自审美和产品偏好。`unknowns/03` 页面把同一组 `review queue` 数据渲染成四种互相差异明显的方向：`dense ops console`、`airy editorial cards`、`kanban / timeline hybrid`、`brutalist mono terminal`。人不必先写完整设计 brief，只要挑出态度和可偷取的细节。

`unknowns/04-toolbar-mock.html` 更进一步，把新的 `frame annotation toolbar` 做成可点击的静态 HTML。页面直接暴露四个布局决策：工具条放在哪里、颜色和描边控制是否常驻、评论抽屉是否

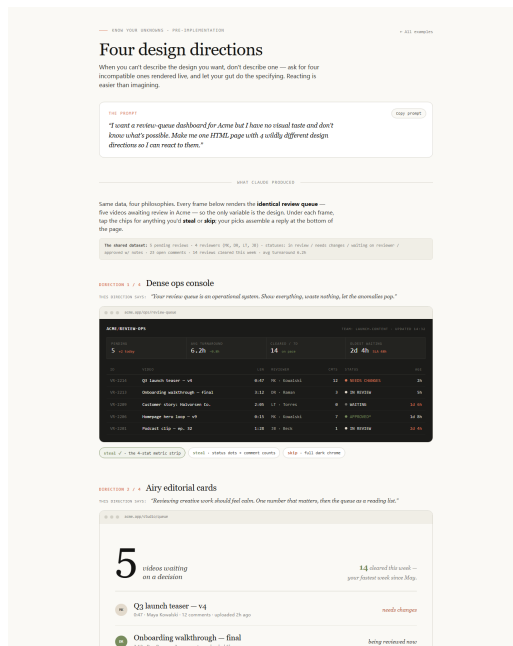


图 7: 同一组 review queue 数据被渲染成四种设计方向，让“我不知道自己喜欢什么”变成可以选择和组合的视觉反馈。

遮挡画面、blur 工具是否进入 v1。这样的 mock 让争论落到具体界面，把抽象形容词转成可点选的判断。

3.4 把方案空间和需求歧义摊开

unknowns/05-churn-brainstorm.html 处理的是方案空间未知。页面要求代理搜索现有代码，并按“最便宜到最雄心勃勃”列出十个干预点。示例里很多方案的重点是把已有但断开的 retention 机制接起来：空项目页没有导入 NewProjectButton，示例项目 flag 没开启，里程碑表存在但客户端没有读取，first comment 的 affordance 藏在时间线上。

unknowns/06-interview.html 处理的是需求歧义。它要求 Claude 一次只问一个问题，并按“答案会多大程度改变架构”排序。这个排序很重要，因为所有问题看起来都能问，真正影响架构的问题应该先被回答。采访工件的目标是收敛决策，并控制讨论长度。

方案空间的判断

当人只知道“问题很粗糙”，适合先要全局选项；当人已经知道功能范围，却说不清关键边界，适合让代理采访。前者扩大视野，后者压缩歧义。

3.5 用语义映射保护移植等价性

unknowns/07-reference-port.html 处理的是参考实现未知。示例要求把 Rust 的 rate limiter 语义移植到 TypeScript API client，但在写代码前先生成 semantics map。页面把 token bucket、lazy refill、integer truncation、decorrelated jitter、retry budget、failure classification 等行为逐项映射到 TypeScript 设计，并标注哪些地方等价，哪些地方只是表面不同。

这类工件适合在移植、重写、替换依赖和跨语言实现时使用。它先证明代理理解参考实现，再进入代码。风险最高的地方往往是“看起来可以直译”的部分，例如时间单调性、整数截断、重试预算、错误分类边界和并发模型。

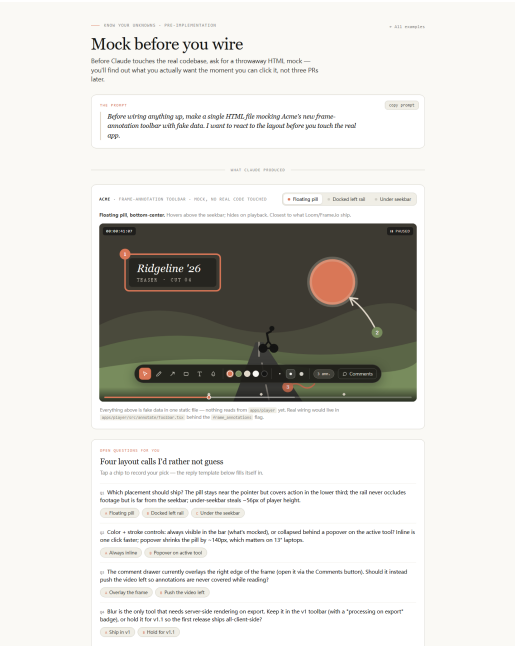


图 8: Toolbar mock 使用假数据和局部交互提前暴露布局决策。页面明确说明真实接线位置和 feature flag，避免把 mock 误认为已实现功能。



图 9: Semantics map 让“照着参考实现改写”变成可审阅的语义确认。确认对象是行为等价性，文件名和语言语法相似性只提供辅助线索。

3.6 把最可能被人修改的决策放到前面

unknowns/08-implementation-plan.html 处理的是计划顺序未知。常规计划会按执行顺序排列，读者很容易先看到机械工作，再错过真正需要人类判断的选择。Tweakable plan 则按“人最可能想改”的顺序组织：数据模型、新类型接口、用户可见流程先出现，机械重构和 plumbing 被折叠到底部。

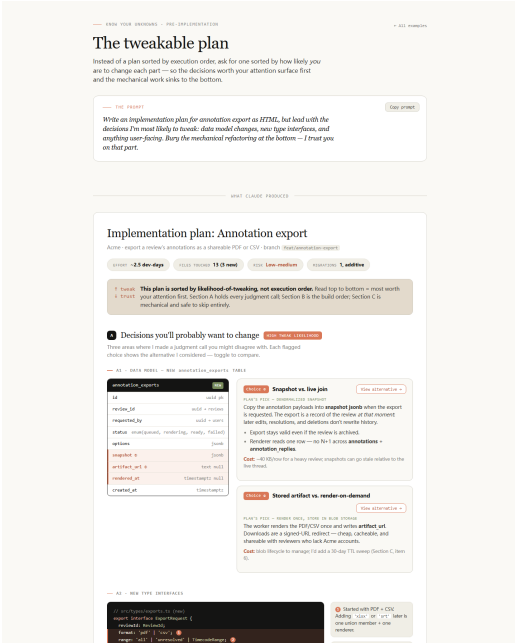


图 10: Tweakable plan 把数据模型、接口和用户体验这些高判断成本决策放在前面，让评审注意力先落到最可能需要修改的地方。

这种工件的核心是重新排序注意力。实践者可以先决定 snapshot 还是 live join，先判断 stored artifact 还是 render-on-demand，先确认 UX flow，再把低争议的重构交给代理执行。

实施前工件的风险

低成本工件可能制造虚假的确定性。假数据、静态交互和示意流程只能暴露偏好与问题，不能证明真实系统已经满足权限、性能、数据一致性和可访问性要求。因此，页面必须清楚标明模拟范围、真实接线位置和仍需验证的边界。

3.7 本章小结

实施前的八类 HTML 工件分别处理不同 unknowns: Blindspot pass 处理地形风险，教学页处理领域词汇，设计方向和 mock 处理隐性偏好，brainstorm 和 interview 处理方案空间与需求歧义，semantics map 处理移植等价性，tweakable plan 处理决策排序。它们的共同作用，是用便宜工件暴露高成本盲区，让真正的代码实现带着更准确的地图进入地形。下一章继续说明代码实现开始后，工件如何记录地图被现实修正的证据。

4 实施中：让偏离计划的现实变成下一轮输入

实施中的核心结论是：计划偏离本身就是高价值材料。真正危险的情况是偏离只留在终端滚动记录、聊天上下文或个人记忆里，下一轮计划仍按旧地图前进。“Implementation notes” 页面把中途发

现拆成可审阅日志：计划原文、示例代码现实、保守选择、待回访问题分别落位，于是现实地形对计划的修正可以被下一轮直接继承。

本章主张

长程代理实施任务时，最需要保留的材料是现实让计划哪里失效、代理选择了什么保守路径、哪些判断必须回到人手里。这类记录把实施过程从一次性执行变成可学习的工程循环。

4.1 实施日志解决的真实问题

实施前的“Tweakable plan”已经把高风险决策提前暴露，例如数据模型、类型接口、用户可见流程和机械重构的排序。可是任何计划都只能覆盖当时已经进入地图的内容。以下 Acme 工程细节均来自示例页面，用于说明工作法。示例代码库包含迁移遗留、已上线工具、权限例外、队列序列化、运维约定和产品承诺，这些内容只有在代理进入代码细节后才会浮现。

“Implementation notes”的提示词要求代理在构建 export feature 时持续维护实施记录：遇到边界情况导致计划偏离时，先选择保守方案，把偏离写入“Deviations”，然后继续推进。这个指令有两层含义：

- 第一，代理可以继续执行，避免每个小发现都打断任务。
- 第二，人的判断权被保留下来，因为每个偏离都有复盘入口。
- 第三，下一次计划会吸收本次暴露的现实约束。

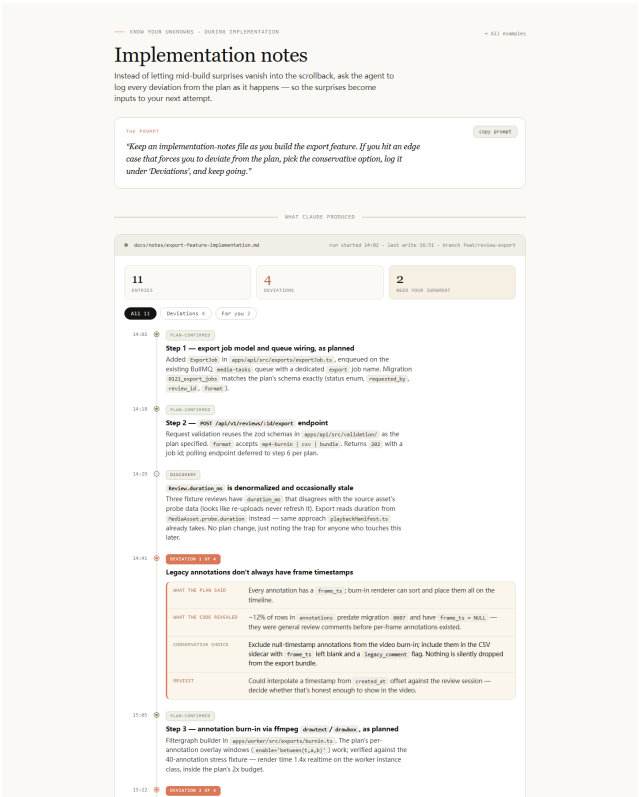


图 11：“Implementation notes”把实施过程拆成已按计划完成、发现、偏离和需人判断四类记录。

页面中的日志从 14:02 到 16:51 记录了 11 个条目，其中 4 个是偏离，2 个需要人的判断。这个比

例很重要：大多数工作仍按计划推进，偏离日志没有把任务变成会议纪要；它只捕获会影响未来决策的部分。

4.2 四格记录法：把偏离从噪声变成证据

“Implementation notes”中每个偏离都采用相同结构：“What the plan said”、“What the code revealed”、“Conservative choice”、“Revisit”。这相当于把一次意外拆成四个可检查问题。

记录栏位	教学含义
计划原文	说明代理原先依据哪张地图行动，避免事后凭印象重写历史。
示例代码现实	说明真实地形出现了什么新约束，例如遗留数据、权限模型或现有工具。
保守选择	说明当前实现如何降低风险，通常优先选择可回滚、少依赖、少承诺的路径。
待回访问题	说明哪些内容属于产品、合规、架构或团队约定判断，需要人或团队确认。

这个结构的价值在于分离事实与判断。比如“遗留 annotations 没有 `frame_ts`”是事实，“排除空时间戳的注释进入视频 burn-in，同时保留在 CSV sidecar”是保守选择，“能否根据 `created_at` 插值到视频时间轴”是需要回访的解释边界。三者混在一段叙述里，评审者很难判断哪里可以批准，哪里需要改计划。

类比：飞行记录仪

实施日志类似工程任务里的飞行记录仪。它不替代飞行员，也不替代路线图；它保存路线和现实之间的偏差。事故复盘、下一次飞行计划和团队训练都依赖这种可回放记录。

4.3 四个偏离展示了四类现实边界

页面中的四个偏离覆盖了常见工程未知。

第一类是遗留数据边界。计划假设每条 annotation 都有 `frame_ts`，示例页面呈现约 12% 的旧数据早于迁移 0087，时间戳为空。保守选择是：视频 burn-in 排除这些无法定位的注释，CSV sidecar 保留它们并标记 `legacy_comment`。这个选择保住了两个原则：视频时间轴不伪造事实，导出包也不静默丢数据。

第二类是运行时表示边界。计划把 `ExportJobPayload.requestedAt` 写成 `Date`，BullMQ 经过 Redis JSON 序列化后实际传给 worker 的是 ISO 字符串。保守选择是把 payload 类型改成 wire format，在 worker 边界解析一次。这里的教训是：跨队列、跨进程、跨网络的数据结构应先按传输形态建模，再在边界恢复领域对象。

第三类是已有能力边界。计划准备增加 `archiver` 依赖，示例页面呈现既有 `zipStream.ts` 工具已经支撑 bulk asset download，并包含 backpressure 处理。保守选择是复用已有工具。这个偏离属于“计划太孤立”：代理原先没有把现有工具库存纳入地图。

第四类是权限语义边界。计划把导出权限设为 review-level ACL，示例页面呈现 `guest_reviewer` 可以查看 review，却有 `can_download_assets = false` 限制。导出源视频会让 guest 间接下载资产。保守选择是所有格式返回 403，并把是否允许 guest 导出纯 CSV 留给产品判断。

常见误区：把偏离当作执行错误

偏离不等于失败。只要日志写清楚原计划、现实证据、临时选择和回访问题，偏离就变成下一轮输入。真正需要警惕的是“代理悄悄修正计划”，导致团队只看到最终代码，看不到关键假设如何改变。

4.4 从实施日志折回下一轮计划

页面结尾把这轮示例实施压缩成三条可复制到下一轮计划的输入：

- 先声明遗留数据 caveat：旧 annotation 可能没有 `frame_ts`，插值和 sidecar 策略应在实施前决定。
- 把队列 payload 规格写成 wire type：跨 BullMQ 的时间字段用 ISO 字符串，并考虑共享 zod codec。
- 增加 prior art 搜索步骤：实施前先查找既有工具，例如 `zipStream.ts` 和 progress channel helper，同时提前处理 ACL 相关产品判断。

这一步完成了从“本轮现实”到“下一轮地图”的转换。计划不再只是人最初的想象，也不只是代理执行后的总结；它成为一份吸收了代码库反馈的更新版工作契约。

可复用实施提示

实施任务可以显式要求代理维护“Implementation notes”：每当计划被代码库现实修正，记录计划原文、代码现实揭示、保守选择和待回访问题；能保守推进的继续推进，需要人决策的单独列为待确认项。

4.5 本章小结

实施阶段的未知来自代码库现实对计划的修正。“Implementation notes”的价值在于把中途发现变成结构化证据：哪些内容按计划完成，哪些发现改变了实现路径，哪些选择只是保守过渡，哪些问题必须回到人或团队。这样一来，下一轮提示、计划和评审可以直接继承本轮暴露的现实约束。下一章转向交付侧，说明这些证据如何被团队审阅、批准和继承。

5 实施后：把个人理解变成团队可接受的交付证据

实施后的核心结论是：交付阶段的未知对象从代码本身转向团队成员。实现者可能已经理解变更，评审者、合并者、安全负责人、设计负责人和产品负责人仍需要证据来判断是否可以接受。“Buy-in doc”和“Quiz before merge”分别处理两类交付未知：别人为什么应该同意，以及实现者是否真的理解自己准备合并的变更。

本章主张

交付阶段要把“个人已经明白”转换成“团队可以审阅、追问、批准和继承”的证据。HTML 工件适合作为这个转换层，因为它能同时承载 demo、spec、风险、测试、问题和交互式理解检查。

5.1 Buy-in doc: 先回答别人会问的问题

“The buy-in doc” 页面的提示词要求把 prototype、spec 和 implementation notes 打包成一份可发到 Slack 的交付文档，并且 “Lead with the demo”。这个顺序很关键：评审者先看到功能如何工作，再看为什么值得发布、风险如何控制、谁需要批准什么。以下 Acme 交付细节均来自示例页面，用于说明工作法。

示例页面中的交付文档围绕 annotation export 展开，开头给出明确请求：四个 sign-off 在周五前完成，feature flag ramp 在周一开始。随后文档用一个 demo 展示选择 annotation、点击 export、得到结果的流程；接着用 pitch 说明用户痛点：reviewer 会把 Acme 里的反馈重复复制到 email、Notion 和剪辑软件，annotation export 让 SRT、CSV、PDF 一键生成。

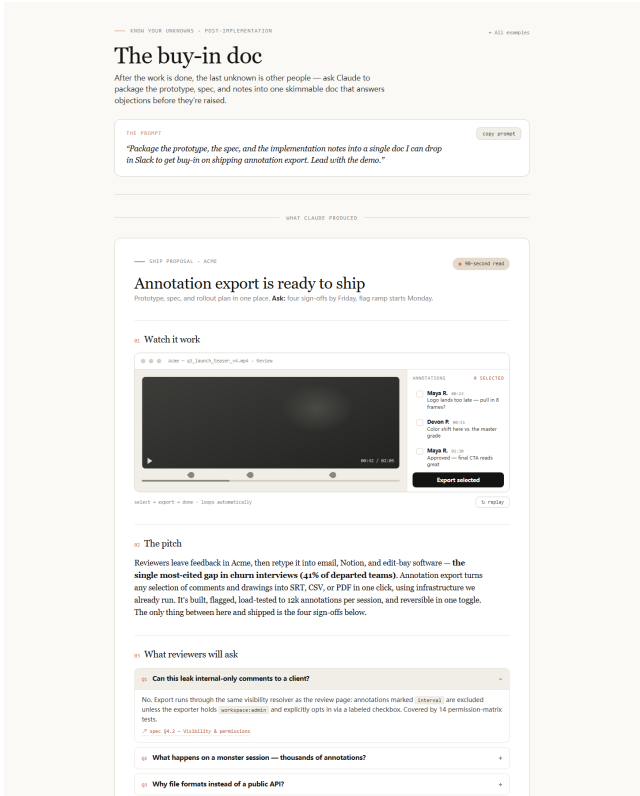


图 12: Buy-in doc 先展示 demo，再集中回答评审者最可能提出的问题。

这个文档的证据层次可以拆成五类：

证据类型	在页面中的表现
用户价值	churn interview 中最常见 gap，annotation export 直接减少重复搬运反馈。
可见功能	demo 展示 select、export、done 的闭环，降低抽象 spec 的理解负担。
技术边界	格式、入口、权限、限制、telemetry、feature flag 都压缩进 spec 表。
风险控制	rollback 一键关闭，worker pool 有 30 秒 job timeout，URL 和文件有过期策略。
责任分配	明确四位 approver 分别审批 shared infra、visibility/audit、modal/error state、flag ramp。

为什么要“先放 demo”

在团队评审里，demo 相当于共享参照物。没有 demo 时，评审者会各自想象功能长什么样；有了 demo，后续关于权限、容量、风险和发布节奏的讨论都围绕同一个对象展开。

5.2 评审者未知：反对意见需要提前显形

“Buy-in doc” 中的 “What reviewers will ask” 模块直接列出评审者问题。例如：

- 内部评论是否会泄露给客户。
- 大型 session 上千条 annotation 时会发生什么。
- 为什么先做文件格式，暂缓 public API。
- 是否引入新基础设施。
- 导出客户内容的 compliance 证据在哪里。

这些问题属于交付材料的核心。实施前的未知主要是“自己还不知道什么”；实施后的未知变成“别人会以什么标准质疑这件事”。好的交付文档把反对意见前置，给出证据、引用 spec、测试结果、权限规则和回滚路径。

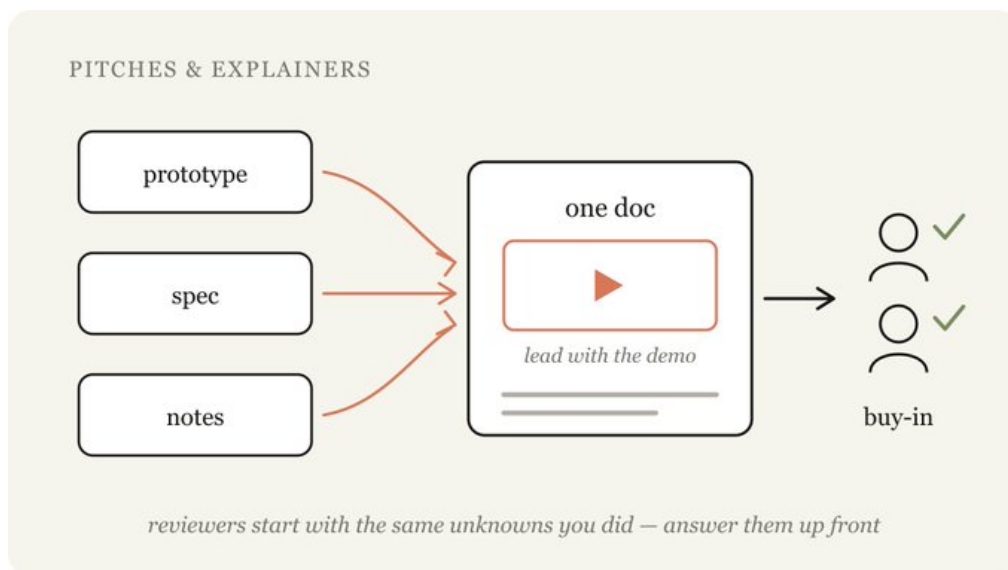


图 13: Buy-in doc 的交付流：demo 建立共同对象，pitch 说明价值，FAQ 与 spec 处理评审阻力。

这个页面也展示了交付文档的边界意识。它保留了 “Known gap”：CSV 中 drawing annotation 会被压平成 bounding box，完整向量仅 PDF 支持，并标为 phase 2。这种已知限制能减少评审成本，因为团队知道当前发布承诺到哪里为止。

5.3 Quiz before merge：把“看过 diff”变成可验证理解

“Quiz me before I merge” 处理另一类交付未知：实现者自己是否真正理解变更。示例页面给出的场景是一个 14 文件 diff，把 review thread 的 clip export 从浏览器端 MediaRecorder 改成服务端 worker 渲染。工件先讲 mental model，再列出三个 diff 浏览很容易漏掉的行为，最后用六个问题验证理解。

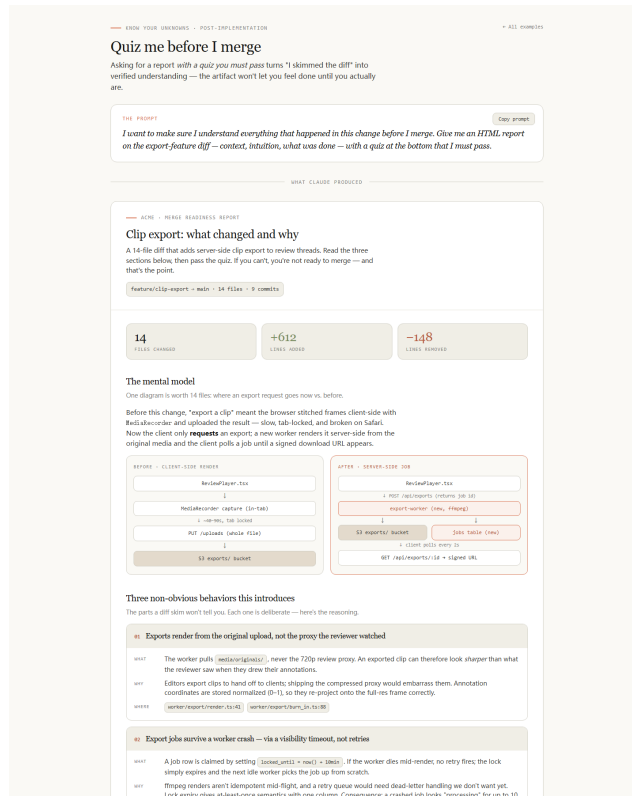


图 14: Quiz before merge 用交互式问题把合并前理解转化为可见结果。

这些 quiz 问题都服务于实战判断。它们对应事故和评审中必须做对的判断：

- 导出为什么可能比 reviewer 看到的 720p proxy 更清晰：worker 使用 full-res original。
- worker OOM 后如何恢复：job 依赖 10 分钟 visibility timeout 被下一位 worker 接手。
- 3 天后的 Slack 链接失效如何处理：S3 signed URL 24 小时过期，对象保留 7 天，重新请求即可。
- guest reviewer session 变更为什么影响 export download：下载接口复用成员校验中间件。
`GET /api/exports/:id -> requireWorkspaceMember`
- “processing 15 分钟”说明什么：超过一次 crash 的自愈窗口，可能是重复失败或 worker 卡住。
- 不同分辨率下 annotation 为什么还能对齐：坐标以 0-1 归一化存储。

合并前的危险信号

如果实现者只能复述“改了哪些文件”，却无法解释新行为、异常恢复、权限继承、过期策略和事故响应，那么 diff 仍然没有被真正消化。此时继续合并会把个人未知转移给值班者和后续维护者。

5.4 状态报告与事故报告：团队继承需要稳定格式

根目录中的 `11-status-report.html` 和 `12-incident-report.html` 展示了另一种交付证据：团队状态与事故复盘。状态报告把一周工程工作压缩成 14 个 merged PR、6 次 deploy、1 个 incident、

3 个 flaky tests fixed，并把 highlights、shipped、velocity、carryover 放在同一页面。事故报告则从 TL;DR 开始，随后给出 timeline、root cause、impact 和 action items。

这两类页面说明，HTML 工件在交付后可以承担“团队记忆”的角色：

- 状态报告让团队知道哪些结果已经完成、哪些风险仍在 carryover。
- 事故报告让团队知道错误如何发生、影响多大、恢复路径是什么、哪些后续动作已有 owner。
- PR writeup 让评审者知道哪里需要重点看，例如 retry/dead-letter 边界、singleton key 和有意排除的范围。

这些页面共同指向同一个原则：交付证据应当面向接收者组织。经理需要进度和风险，评审者需要重点文件和测试，on-call 需要 root cause 与 action items，产品和安全需要 sign-off 边界。

5.5 本章小结

实施后的未知主要落在团队接收侧。“Buy-in doc”把 demo、pitch、spec、风险、rollback 和 sign-off 打包成别人可以批准的材料；“Quiz before merge”把实现者的理解变成可验证结果；状态报告、PR writeup 和事故报告则让团队可以继承进展、风险和后续动作。交付质量同时取决于代码完成度和证据可接受度。最后一章会把实施前、实施中、实施后的工件合成一套可复制模板。

6 可迁移工作法：从案例库到团队模板

可迁移结论是：“Know your unknowns”与“HTML Effectiveness”的案例可以压缩成团队模板。模板的核心规则很简单：先判断未知类型，再选择最低成本的 HTML 工件；先让人能反应，再让代理改真实系统；先把个人发现写成证据，再把证据沉淀进下一轮流程。

最终结论

HTML 工件的意义在于让未知可以被指认、讨论、修正和继承。它把提示词、代码库现实、审查标准和团队偏好放到一个可看的对象里，使人可以更早指出偏差，也使后续执行有更清晰的输入。

6.1 从四类未知到三阶段流程

补充材料把未知分为四类：“Known knowns”是已经写进提示词的目标和约束；“Known unknowns”是人知道还没决定的问题；“Unknown knowns”是人不会主动写下、看到方案才会识别的偏好和经验；“Unknown unknowns”是完全没有意识到的盲区。[unknowns/index.html](#) 又把案例分为实施前、实施中、实施后三个阶段。

两组分类可以合在一起使用。四类未知回答“要找什么”，三阶段流程回答“何时找”。组合后，团队就能把零散案例变成工作台。

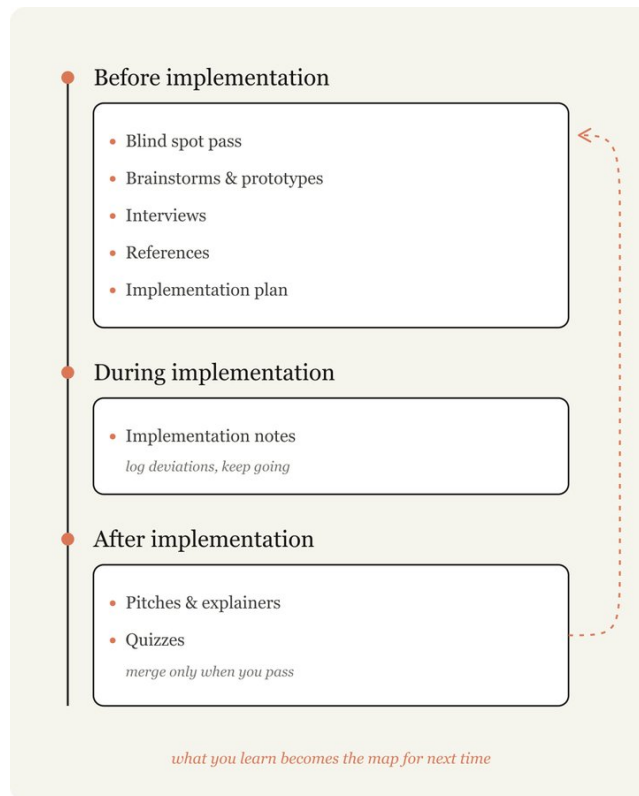


图 15: 从实施前、实施中到实施后，HTML 工件的目标逐步从发现未知转向记录偏离和交付证据。

阶段	主要未知	适合的工件
实施前	领域词汇、设计偏好、架构边界、参考语义、需求歧义	Blindspot pass, explainer, design directions, mock, interview, semantics map, tweakable plan。
实施中	计划与代码现实的差距	Implementation notes, 尤其是计划原文、代码现实揭示、保守选择、待回访问题。
实施后	reviewer、合并者、值班者和后续维护者的接受标准	Buy-in doc, Quiz before merge, PR writeup, status report, incident report。

6.2 选择规则：按未知类型选工件

团队落地时可以使用下面的选择规则。

当前信号	优先工件	判断标准
陌生代码区、历史迁移、权限边界不清楚	Blindspot pass	先让代理扫描代码、提交历史和迁移痕迹，输出 unknown unknowns 与更好的实施提示。
缺少领域词汇或专业判断框架	Explainer	先补概念、术语和可调参数，让人能写出更精确的提示。
审美、布局、交互偏好说不清	Four directions 或 mock	先做可看的低成本页面，让人通过反应表达 unknown knowns。
已有参考实现或参考产品	Semantics map	先证明代理理解行为等价性、边界和不可直译处，再进入移植。
需求歧义会改变架构	Interview	让代理按架构影响排序提问，逐项收敛决策。
计划已经存在，仍可能被人调整	Tweakable plan	把最可能被改的 schema、接口和用户可见决策放在前面。
实施中出现现实修正	Implementation notes	记录偏离和保守选择，把发现折回下一轮计划。
准备让团队批准发布	Buy-in doc	用 demo、FAQ、spec、风险、rollback 和 sign-off 回答评审者问题。
准备合并复杂 diff	Quiz before merge	用问题验证 mental model、异常路径、权限继承和事故响应理解。

选择规则的直觉

如果人说不清“想要什么”，先做可反应的工件；如果代码库可能藏着历史和边界，先做发现型工件；如果已经实现完，先做可接受和可继承的工件。选择依据是未知的位置，页面形式只是承载方式。

6.3 边界：HTML 工件不能替代真实验证

HTML 工件的能力来自低成本、可视化和可交互，也有明确边界。

边界条件

HTML 工件可以暴露意图、偏好、结构和证据；它不能证明生产代码正确、权限安全、性能达标或合规承诺成立。最终仍需要测试、代码审查、运行时验证、日志、监控和真实用户反馈。

落地时需要注意五个限制：

- 原型可能欺骗人。静态页面可以让流程看起来顺畅，真实数据、加载状态、失败路径和权限分支仍需验证。
- 截图可能过期。代码库或产品规则变化后，旧 artifact 应标记来源和更新时间。
- 交互不能覆盖所有状态。复杂系统需要把关键状态写进 spec、测试和通过条件。
- 敏感信息需要清理。导出给团队或客户的 HTML 需要移除 secret、真实用户数据或无关日志。

- 审批责任不能下放给页面。页面可以组织证据，批准仍由对应 owner 承担。

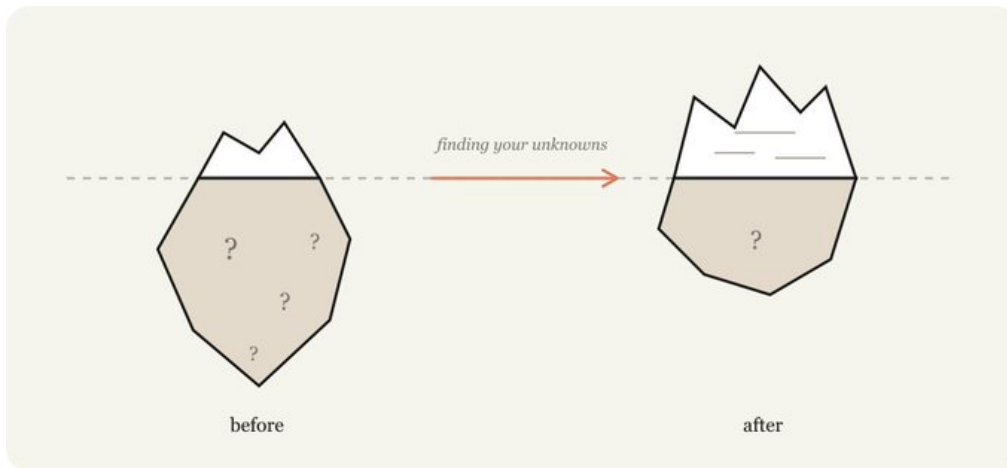


图 16: 未知像冰山：可见需求只是表层，历史约束、审查标准、团队偏好和运行环境常在水面以下。

6.4 团队模板：把案例沉淀成可复制入口

团队可以把这些页面沉淀成一组模板，每个模板包含固定输入、固定输出和退出条件。

模板字段	需要写清楚的内容	例子
适用场景	何时使用这个模板，何时直接跳过。	“陌生模块改动前使用 blindspot pass；小型文案调整可跳过。”
输入材料	代理可读的文件、目录、参考页面、约束和禁止范围。	<code>services/auth/</code> 、历史 commits、参考实现、设计截图。
输出格式	页面必须包含哪些区块。	发现清单、风险等级、建议提示词、待人决策。
人类动作	人看完 artifact 后需要做什么。	选择设计方向、批准权限策略、回答架构问题。
退出条件	什么时候可以进入代码修改或合并。	高风险问题已决定，open questions 有 owner，理解 quiz 通过。

模板化的关键不在于统一视觉风格，而在于统一决策结构。所有模板都应让读者快速看到：结论是什么，证据在哪里，哪些内容需要人判断，哪些内容已经足够自动化。

6.5 落地检查表

团队准备把 HTML 工件引入长程代理 workflow 时，可以用下面的检查表。

- 明确工件的接收者：自己、工程评审者、设计、产品、安全、值班者或客户。
- 写清 source scope：代理读取了哪些目录、diff、HTML、截图、日志或参考实现。
- 声明禁止范围：不读取敏感数据，不改真实代码，不把示例 Acme 当作真实企业事实。
- 先给结论：页面开头说明建议、风险或需要决策的事项。
- 保留可追溯证据：路径、文件、接口、时间、指标、测试结果、owner。

- 区分事实、判断和建议：代码现实揭示属于事实，保守选择属于判断，下一步属于建议。
- 给出人类决策入口：按钮、清单、sign-off、问题列表或可复制回复。
- 记录刷新触发条件：代码改动、需求变化、权限模型变化、事故复盘、评审意见。
- 把通过条件写清楚：例如 open questions 已关闭、quiz 通过、review focus 已覆盖、rollback 已明确。

团队采用原则

先把 HTML 工件用在高不确定、高返工成本、高沟通成本的任务上。低风险、低歧义、单文件机械修改可以直接进入实现。模板的目标是降低认知和返工成本，避免给简单任务增加仪式。

6.6 本章小结

从案例库到团队模板的迁移，应围绕未知类型和工作阶段展开。实施前用 HTML 暴露领域、偏好、架构和参考语义；实施中用日志保存计划偏离；实施后用 buy-in、quiz、PR writeup、状态报告和事故报告把个人理解转换为团队证据。只要团队坚持 source scope、边界、owner、刷新条件和通过标准，HTML 工件就能从一次性演示变成可复用工作法。

A 来源与边界

本文依据随文归档的本地资料包编写。主要来源包括补充网页正文、unknowns 子站 11 个 HTML 页面，以及根目录 20 个 html-effectiveness 示例页面。补充网页包含平台导航、脚本资源和自动翻译残留；正文只吸收与 Finding Your Unknowns 方法有关的内容。unknowns 与 html-effectiveness 页面中的 Acme 场景被视为示范工件，用于讲解工作法。

来源组	本文中的用途
补充网页 supplement-field-guide-fable-finding-your-unknowns.html	解释四类未知、地图与地形、三阶段工作法，以及 Fable 语境下为什么要先暴露 unknowns。
unknowns/index.html	作为 11 个 demo 的总览，说明实施前、实施中、实施后的分类。
unknowns/01..08	说明实施前如何发现盲区、补足词汇、显化偏好、确认语义和排序决策。
unknowns/09	说明实施中如何记录偏离、保守选择和待回访问题。
unknowns/10..11	说明实施后如何争取团队 buy-in，并验证合并前理解。
根目录 01..20 HTML pages	作为 HTML 工件能力的横向案例库，覆盖探索、设计、代码理解、原型、图表、报告和编辑器。